

AI-NATIVE GROCERY MERCHANT

Document 09 — Razorpay Integration Specification

Development-only specification • Locked MVP • Test Mode order creation, checkout, verification, webhooks and payment integrity

Payment principle: Razorpay executes an already-authorized transaction. It is never the component that decides whether the AI is allowed to spend.

Document status: LOCKED DEVELOPMENT SPECIFICATION

Environment: Razorpay Test Mode only for the MVP/buildathon implementation.

Financial chain: User selection → fresh quote → deterministic policy ALLOW → Razorpay order → checkout → server verification/webhook → final audit.

1. Purpose and Scope

This document defines the Razorpay integration boundary for the AI-Native Grocery Merchant MVP. It specifies how an authorized basket becomes a Razorpay Test Mode order, how checkout is launched, how payment state is verified, how webhooks are validated and processed, and how duplicate or failed payment events are handled.

The integration must preserve the central architecture rule: the AI reasons and proposes, the policy engine authorizes, Razorpay executes, and the backend verifies and records the result.

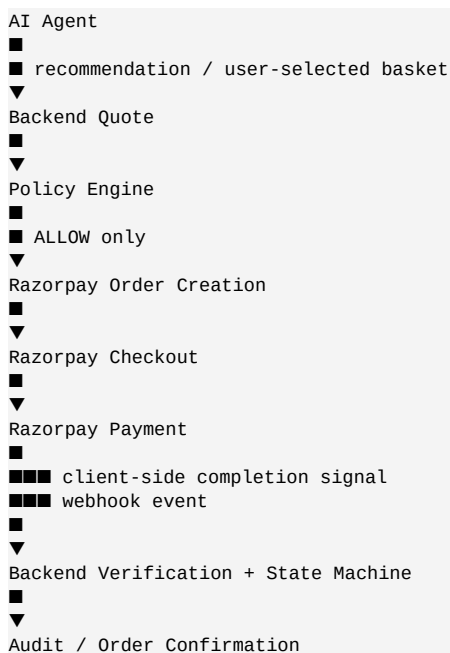
1.1 In scope

- Razorpay Test Mode configuration.
- Server-side order creation.
- Mapping authorized basket amount to Razorpay amount.
- Checkout handoff.
- Payment response handling.
- Server-side payment verification.
- Webhook endpoint and signature verification.
- Webhook idempotency and duplicate handling.
- Order/payment state transitions.
- Audit correlation.

1.2 Explicitly out of scope

- Production/live-money payments.
- Autonomous refunds or payouts.
- Banking operations.
- Payment-method-specific custom integrations beyond the MVP checkout flow.
- Razorpay marketplace/platform settlement features.
- Full production payment operations.

2. Razorpay Role in the Architecture



Razorpay is downstream of authorization. No AI-generated action can directly create a payment order.

3. Environment and Credentials

3.1 Test Mode

All development and buildathon transactions must use Razorpay Test Mode credentials. Test transactions do not move real money.

```
Environment variables
-----
RAZORPAY_KEY_ID=<test-key-id>
RAZORPAY_KEY_SECRET=<test-key-secret>
RAZORPAY_WEBHOOK_SECRET=<webhook-secret>
RAZORPAY_ENV=test
```

3.2 Secret handling

- Never expose RAZORPAY_KEY_SECRET to the browser.
- Never expose webhook secrets to the AI agent.
- Never commit credentials to source control.
- Use environment variables or the deployment secret manager.
- Do not log secrets or complete credential values.
- The browser receives only the public key identifier required by the checkout integration.

The exact credential names may be adapted to the implementation, but the separation between public and secret material is mandatory.

4. Order Amount Convention

Razorpay order amounts are represented in the smallest currency unit. For INR, ₹299 is sent as 29900.

```
UI:
₹874

Backend:
final_payable_minor = 87400

Razorpay order:
amount = 87400
currency = "INR"
```

4.1 Amount source

The amount sent to Razorpay must originate from the authoritative backend quote that passed policy authorization. It must never come from the frontend, LLM, or an unverified basket display.

4.2 Amount invariant

```
razorpay_order.amount
==
authorized_policy.final_payable_minor
==
fresh_authoritative_quote.final_payable_minor
```

If these values do not match, the backend must fail closed and re-quote/re-authorize before creating the order.

5. Pre-Payment Authorization Gate

The payment integration may only be reached after the following sequence.

1. User selects basket.
2. Backend validates selection ownership.
3. Backend refreshes product/stock/incentive state.
4. Backend calculates fresh quote.
5. Policy engine evaluates mandate + quote.
6. Policy result = ALLOW.
7. Backend creates Razorpay order using authorized amount.

8. Persist application order + Razorpay order ID.
9. Launch checkout.

5.1 No bypass

- No checkout route may accept an arbitrary amount from the browser.
- No checkout route may proceed without policy ALLOW.
- No model tool may create a Razorpay order.
- No previously approved stale quote may be reused after a material state change.

6. Application Order Creation

6.1 Internal endpoint

```
POST /api/v1/checkout
Idempotency-Key: checkout-selection-001

{
  "selection_id": "selection-001",
  "policy_decision_id": "pd-001"
}
```

The endpoint must load the selected basket, policy decision and mandate from server-side state. It should not accept an amount field from the client.

6.2 Pre-conditions

Check	Required
Authenticated user	Yes
Selection belongs to user/session	Yes
Basket is current	Yes
Policy decision belongs to basket/mandate	Yes
Policy decision = ALLOW	Yes
Authorization is fresh/valid	Yes
Authorized amount available	Yes
Idempotency key valid	Yes

7. Razorpay Order Creation

The backend creates the Razorpay order using the authorized final payable amount.

```
Conceptual request:
{
  "amount": 87400,
  "currency": "INR",
  "receipt": "order-001"
}
```

7.1 Required persistence

After successful Razorpay order creation, persist at minimum:

- Application order_id.
- Razorpay order ID.
- Authorized amount.
- Currency.
- Policy decision ID.

- Basket ID.
- Mandate ID.
- Current order status.
- Creation timestamp.

7.2 Receipt

Use an application-generated receipt/reference that uniquely correlates the Razorpay order to the internal order. Do not expose unnecessary user data in the receipt.

8. Application Order State

```

CREATED
↓
PAYMENT_PENDING
■■■ PAYMENT_VERIFIED
■■■ PAYMENT_FAILED
■■■ PAYMENT_CANCELLED
■■■ PAYMENT_EXPIRED (if applicable)

```

The exact terminal state names can be implemented consistently with the application model, but the system must distinguish pending, failed and verified outcomes.

8.1 State authority

Only backend payment processing and verified provider events may change payment state. The frontend and AI may report signals but cannot directly set VERIFIED.

9. Checkout Handoff

The frontend receives the public information required to initialize Razorpay Checkout after the backend has created an authorized order.

```

{
  "order_id": "order-001",
  "razorpay_order_id": "order_test_123",
  "amount_minor": 87400,
  "currency": "INR",
  "razorpay_key_id": "<public-test-key>"
}

```

9.1 Browser restrictions

- The browser must use the Razorpay order created by the backend.
- The browser must not create an independent Razorpay order.
- The browser must not change the amount.
- The browser's payment-success callback is not final authorization of payment success.
- The UI should display payment state returned by the backend.

10. Client-Side Payment Completion

After checkout, Razorpay may provide identifiers/signatures to the client. The application may send these values to the backend for verification.

```

POST /api/v1/payments/verify

{
  "order_id": "order-001",
  "razorpay_order_id": "order_test_123",
  "razorpay_payment_id": "pay_test_123",
  "razorpay_signature": "<signature>"
}

```

The backend validates the response using the appropriate Razorpay verification mechanism and its server-side secret. The client-provided result is evidence to verify, not a trusted final state.

11. Payment Verification

11.1 Verification principle

Payment should be marked verified only after server-side verification establishes that the payment belongs to the expected Razorpay order and the cryptographic/provider checks succeed.

11.2 Amount integrity

- Expected application order amount must match the authorized order amount.
- Verified payment must correspond to the expected Razorpay order.
- A mismatched amount must not become VERIFIED.
- Unexpected order/payment relationships must be rejected or quarantined for investigation.

11.3 Verification result

```
{
  "payment_id": "payment-001",
  "order_id": "order-001",
  "status": "VERIFIED",
  "razorpay_payment_id": "pay_test_123",
  "amount_minor": 87400,
  "verified_at": "2026-09-02T12:04:00Z"
}
```

12. Webhook Integration

Webhooks provide server-side notification of payment lifecycle events and must be treated as an independent provider event stream.

12.1 Endpoint

```
POST /api/v1/webhooks/razorpay
Content-Type: application/json
X-Razorpay-Signature: <signature>
```

12.2 Processing pipeline

```
Receive raw request
↓
Verify webhook signature
↓
Identify event
↓
Check event idempotency
↓
Map provider IDs to internal order/payment
↓
Validate state transition
↓
Persist state
↓
Write audit event
↓
Return accepted response
```

13. Webhook Signature Verification

Webhook authenticity must be verified using the configured Razorpay webhook secret and the exact raw request body as required by the provider's signature-verification mechanism.

13.1 Implementation rules

- Use the raw request payload for signature verification before parsing/mutating state where required by the framework.
- Use the server-side webhook secret.

- Reject invalid signatures.
- Do not accept a user-provided secret.
- Do not process financial state before verification.
- Log a safe verification failure event without exposing secrets.

The webhook secret is distinct from the API key secret and must be stored server-side.

14. Webhook Event Handling

Event class	Application behavior
Payment authorized/captured equivalent	Move payment toward VERIFIED only after required validation.
Payment failed	Set PAYMENT_FAILED if compatible with current state.
Payment pending	Keep PAYMENT_PENDING.
Order/payment state event	Update only if the event is valid for the internal state machine.
Unknown event	Record safely and do not mutate financial state blindly.
Duplicate event	Return idempotent success/no-op after prior processing.

Exact event names should follow the Razorpay event types enabled for the project and current provider documentation. The application must not assume that event ordering is guaranteed.

15. Webhook Idempotency

Duplicate webhook deliveries are expected. Processing must be idempotent.

15.1 Event record

```
razorpay_webhook_event
-----
event_id VARCHAR UNIQUE
event_type VARCHAR
payload_hash VARCHAR NULL
received_at TIMESTAMP
processed_at TIMESTAMP NULL
processing_status ENUM
error_code VARCHAR NULL
```

15.2 Processing rule

```
if event_id already processed:
    return success / no-op

else:
    validate signature
    validate event
    apply valid state transition
    mark event processed
```

If the provider does not expose a suitable unique event identifier for a specific event flow, use a safe deduplication strategy based on stable provider identifiers and event characteristics.

16. Event Ordering and Reconciliation

Provider events may not always arrive in the same sequence as client-side signals. The application must use a state machine rather than blindly overwriting status.

16.1 Rules

- A late failure event must not overwrite a verified payment without a defined valid transition.
- A duplicate success event must not create another payment.
- A payment cannot become VERIFIED merely because the frontend reported success.

- For critical uncertainty, query the provider/backend state rather than guessing.
- Persist raw provider identifiers needed for reconciliation.

17. Payment State Machine

```

PAYMENT_PENDING
■
■■■ provider confirms success ■■→ VERIFIED
■
■■■ provider confirms failure ■■→ FAILED
■
■■■ timeout/cancellation ■■■■■■■■→ CANCELLED/EXPIRED

VERIFIED
■
■■■ no ordinary backward transition

FAILED
■
■■■ new payment attempt may create a new payment flow,
subject to application rules

```

A verified payment should be treated as terminal for the current transaction unless a separately specified post-payment process exists. Refund behavior is out of scope for this MVP.

18. Amount Reconciliation

Every layer must agree on the authorized amount.

Layer	Amount
Selected basket quote	Fresh final payable
Policy decision	Authorized final payable
Application order	Authorized amount
Razorpay order	Same amount in minor units
Verified payment	Expected payment amount

18.1 Mismatch behavior

Any mismatch is a failure condition. The application must not silently adjust the authorized amount. It should stop the flow, record the discrepancy and require safe revalidation or reconciliation.

19. Idempotent Checkout Creation

The checkout endpoint must protect against double-clicks, browser retries, network retries and duplicate frontend submissions.

```

Request A:
Idempotency-Key = checkout-selection-001
→ creates order-001 / Razorpay order-test-123

Request A retry:
same key + same request
→ returns order-001

Same key + materially different request
→ IDEMPOTENCY_CONFLICT

```

19.1 Required invariant

One logical checkout attempt must not create multiple application orders or multiple Razorpay orders because of a retry.

20. Failure Handling

Failure	Required behavior
Razorpay order creation fails	Keep application order/payment path non-successful; return provider-safe error; retry only with idempotency protection.
Checkout cancelled	Keep order in appropriate pending/cancelled state; do not claim payment.
Payment fails	Mark FAILED after verified provider/backend state.
Webhook signature invalid	Reject and do not mutate payment state.
Duplicate webhook	No-op after idempotency check.
Payment amount mismatch	Do not verify; flag/reconcile.
Unknown provider event	Record without unsafe state mutation.
Network timeout after order creation	Use idempotency/reconciliation rather than blindly creating another order.
Policy becomes stale before checkout	Do not create order; re-quote and re-authorize.
Application crash during processing	Persist intermediate state transactionally and reconcile on restart.

21. Graceful Policy Failure Before Razorpay

A policy denial must prevent any Razorpay order from being created.

```
Mandate max = ₹800
Selected basket final payable = ₹850

Policy:
DENY / MAX_SPEND_EXCEEDED

Result:
- no Razorpay order
- no payment attempt
- audit denial
- optimizer may re-optimize
- new quote + policy check required
```

This is the primary financial safety demonstration for the buildathon: an intelligent agent can recover from a constraint violation without bypassing authorization.

22. Application-to-Razorpay Data Mapping

Application field	Razorpay mapping
final_payable_minor	Order amount
currency	Order currency
order_id	Receipt/reference metadata where appropriate
application order state	Internal only
mandate_id	Internal audit/reference only
basket_id	Internal audit/reference only
policy_decision_id	Internal authorization reference

Do not place sensitive mandate data or unnecessary personal information into provider metadata. Internal identifiers are primarily for application reconciliation and audit.

23. API Security

- Razorpay API credentials are server-only.
- Webhook secret is server-only.
- Checkout public key may be sent to the browser.
- All internal payment endpoints require authenticated application context.

- Order/payment IDs must be bound to the current user/session.
- Do not trust client-supplied amount or status.
- Validate every external provider identifier before updating internal state.
- Use HTTPS outside local development.
- Do not log full payment signatures or secrets.

24. Audit Requirements

The payment integration must produce an auditable chain from policy authorization through provider execution and final verification.

24.1 Required events

Event	Minimum information
POLICY_ALLOWED	Policy decision ID, basket, mandate, authorized amount
RAZORPAY_ORDER_CREATED	Internal order ID, Razorpay order ID, amount
CHECKOUT_STARTED	Internal order/reference
PAYMENT_VERIFY_REQUESTED	Order and provider IDs, no secrets
PAYMENT_VERIFIED	Payment ID, amount, verification timestamp
PAYMENT_FAILED	Order/payment ID and safe failure code
WEBHOOK_RECEIVED	Event ID/type and receipt timestamp
WEBHOOK_PROCESSED	Event ID and resulting state
AMOUNT_MISMATCH	Expected vs observed amount, safe identifiers

25. Environment Configuration

```
# Development / Buildathon
RAZORPAY_ENV=test
RAZORPAY_KEY_ID=...
RAZORPAY_KEY_SECRET=...
RAZORPAY_WEBHOOK_SECRET=...

# Application
APP_BASE_URL=...
API_BASE_URL=...
```

Do not commit .env files containing credentials. Provide a documented .env.example with placeholder values.

26. Local Development Webhook Strategy

Because provider webhooks must reach a publicly accessible server endpoint, local development may require a secure tunneling or equivalent development setup supported by the chosen environment. The implementation should document the configured webhook URL and secret without committing credentials.

For the buildathon demo, the webhook path should be tested before the final demonstration so that payment verification and audit updates can be shown reliably.

27. Test Matrix

Scenario	Expected result
Valid ALLOW + Test Mode order	Razorpay order created with exact authorized amount
DENY before checkout	No Razorpay order created
Client sends lower amount	Ignored; backend uses authorized amount
Client sends higher amount	Ignored/rejected

Scenario	Expected result
Policy quote and order amount mismatch	Fail closed
Duplicate checkout request	Same application/Razorpay order result
Payment success callback	Backend verification required
Valid webhook	Payment state updated according to state machine
Invalid webhook signature	Rejected; no state mutation
Duplicate webhook	No duplicate payment/state transition
Out-of-order events	State machine prevents invalid overwrite
Payment amount mismatch	Not VERIFIED; reconciliation required
Razorpay API timeout	Safe retry/reconciliation; no duplicate order
Expired mandate before checkout	No order; policy DENY
Stock changes before checkout	Fresh quote/re-authorization required
Test Mode environment	No live-money credentials or endpoints

28. Demo Acceptance Flow

```

1. User enters:
"Pasta for 4, budget █1000"

2. Agent researches and optimizes.

3. UI shows:
Best Value
Best Quality
AI recommendation

4. User selects basket.

5. Backend creates fresh quote.

6. Policy engine:
ALLOW

7. Backend creates Razorpay Test Mode order.

8. Razorpay Checkout opens.

9. Test payment is completed.

10. Backend verifies payment / processes webhook.

11. UI shows verified order state.

12. Audit trail shows:
user goal
→ optimization
→ recommendation
→ user selection
→ policy ALLOW
→ Razorpay order
→ payment verification

```

28.1 Failure demonstration

A second demo path should intentionally create a basket above the mandate limit. Policy must return DENY / MAX_SPEND_EXCEEDED, no Razorpay order should be created, and the agent should recover by producing a feasible basket.

29. Operational Logging

- Log request ID and application order ID.
- Log Razorpay order/payment identifiers when safe.

- Log state transitions and policy decision references.
- Log webhook event IDs and processing outcomes.
- Never log API secrets, webhook secrets or payment signatures.
- Avoid logging complete personal/payment payloads when references are sufficient.

30. Definition of Done

- Razorpay Test Mode credentials are configured securely.
- Server-side order creation is implemented.
- Razorpay amount exactly matches the authorized final payable.
- No checkout can occur without policy ALLOW.
- Client cannot control the payment amount.
- Application order references the policy decision and basket.
- Razorpay order ID is persisted.
- Checkout handoff uses the backend-created order.
- Payment completion is server-verified.
- Webhook signature validation is implemented.
- Webhook processing is idempotent.
- Payment/order state transitions are validated.
- Amount mismatches fail closed.
- Duplicate checkout attempts do not create duplicate orders.
- Payment failure is never reported as success.
- Policy denial prevents Razorpay order creation.
- Audit events cover authorization, order creation and payment verification.
- Buildathon demo can show both successful and graceful-failure paths.
- No production/live-money functionality is introduced.

31. Related Development Documents

Document	Relationship
01 — Development Master Spec	Locked product and engineering baseline
02 — Functional Requirements	Payment-related functional behavior
03 — System Architecture	Payment boundary and trust model
04 — AI Agent Specification	Agent restrictions around payment
05 — Optimization Decision Specification	Basket and incentive output consumed before payment
06 — Policy & Mandate Specification	Authorization gate before Razorpay
07 — Data Model Specification	Order/payment/audit persistence
08 — API Contract Specification	Checkout, payment and webhook API contracts
10 — Testing & Acceptance Specification	End-to-end payment validation